

Die KI-Komponente des Cocktail-Mixers

Gehirnzellen Massaker – wie die kuenstliche Intelligenz im Projekt funktioniert

Im Projekt stecken **zwei getrennte KI-Bausteine**, die nacheinander greifen. Der erste erzeugt eine Auswahl an Cocktails, der zweite entscheidet, welchen davon der Verlierer trinken muss. Beide laufen vollstaendig auf dem Handy – ohne Server, ohne API-Schluessel und ohne Internetverbindung zur Laufzeit.

- **Rezept-Generierung** – ein Sprachmodell (LLM) erfindet aus den vier eingefuellten Getraenken passende Cocktails.
- **Foto-Matching** – Googles ML Kit analysiert das Selfie des Verlierers und waehlt daraus den passenden Cocktail aus.

Teil 1 – Cocktails aus den Zutaten generieren (On-Device-LLM)

Die Idee

Der Nutzer traegt ueber das Overlay „*What's in the box*“ ein, welche vier Getraenke gerade an den vier Pumpen haengen (z. B. Mango, Limette, Gin, Soda). Daraus soll die KI kreative Cocktails vorschlagen, die sich **nur aus genau diesen vier Zutaten** mischen lassen.

Das Modell

Wir benutzen **Gemma**, ein kleines, offenes Google-Sprachmodell (rund 0,5 GB, 1 Milliarde Parameter), ueber das Flutter-Paket **flutter_gemma**. Das Entscheidende: Das Modell laeuft **komplett lokal auf dem Handy** – kein Server, kein API-Schluessel, keine laufenden Kosten. Es wird einmalig als Datei mitgeliefert.

Der Ablauf

- **Prompt bauen:** Aus den vier Zutaten wird eine Anweisung in natuerlicher Sprache an das Modell gebaut – sinngemaess: „Du bist ein kreativer Barkeeper mit genau vier Pumpen. Erzeuge 4–6 Cocktails, die sich nur aus diesen Zutaten mischen lassen. Antworte ausschliesslich mit gueltigem JSON.“
- **Modell antwortet:** Gemma erzeugt eine Liste von Cocktails als JSON-Text – je mit Namen, Beschreibung, Stimmungs-Tags, Pumpenmengen und einem Verfeinerungs-Tipp.
- **Antwort aufräumen (Sanitizer):** die wichtigste technische Massnahme – siehe unten.

Der Sanitizer – das Herzstueck

Ein Sprachmodell haelt sich nie zu 100 % an Vorgaben: mal packt es Erklaertext drum herum, mal erfindet es einen Tag, mal gibt es unrealistische Mengen an. Deshalb laeuft die Rohantwort durch eine strenge Nachbearbeitung, die alles wieder in ein sauberes, hardware-taugliches Format zwingt: JSON aus Fliesstext/Markdown herausschneiden · jede Pumpenmenge auf 0–80 ml begrenzen und die Gesamtmenge auf max. 250 ml herunterskalieren · mindestens zwei Pumpen benutzen · erfundene Tags verwerfen (nur erlaubte zulassen) · sicherstellen, dass der Name eine echte Zutat enthaelt · die Liste auf max. 6 Cocktails kappen.

Sicherheitsnetz: kann nie abstuerzen

Wenn das Modell nicht geladen werden kann, die Plattform es nicht unterstuetzt oder zu wenige brauchbare Cocktails herauskommen, schaltet das System **automatisch auf einen deterministischen Mock-Generator** um. Der erfindet ohne KI plausible Cocktails aus denselben Zutaten – nach festen Regeln und immer gleich fuer dieselbe Eingabe. So funktioniert die App auch offline, im Testmodus und in den automatischen Tests zuverlaessig.

Das Ergebnis wird als **Pool** gespeichert (samt Pumpen-Belegung), ueberlebt einen App-Neustart und wird nur neu generiert, wenn sich die Zutaten tatsaechlich aendern.

Teil 2 – Den passenden Cocktail per Selfie auswaehlen (Google ML Kit)

Nach dem Schere-Stein-Papier-Match wird nicht zufaellig ein Cocktail gewaehlt, sondern der Verlierer bekommt einen, der zu seinem Gesichtsausdruck „passt“. Das macht **Google ML Kit** – eine On-Device-Bild-KI, ebenfalls ohne Server.

Schritt A – Bildanalyse

- **Gesichtserkennung** liefert eine Laechel-Wahrscheinlichkeit, wie weit die Augen geoeffnet sind und die Kopfneigung/-drehung.
- **Bild-Labeling** liefert Schlagwoerter zum Bild (Farben, Objekte, Stimmung – z. B. „dark“, „tropical“, „bright“).
- Daraus wird eine grobe **Emotion** abgeleitet: viel Laecheln → *happy*, mittel → *neutral*, wenig → *sad*.

Schritt B – Stimmung in Gewichte uebersetzen

Die Merkmale werden in **Gewichte pro Stimmungs-Tag** umgerechnet. Beispiele:

- froehliches Gesicht → verstaerkt *happy, fresh, light, colorful, playful*
- neutraler Ausdruck → *sophisticated, calm, warm, classic*
- ernster/trauriger Ausdruck → *dark, serious, mysterious, intense*
- weit offene Augen → *energetic, bold, adventurous*; gedrehter Kopf → *confident, bold*
- Bild-Labels wie „tropical“ oder „wood“ verstaerken die entsprechenden Tags

Schritt C – Cocktail auswaehlen

Jeder Cocktail aus dem generierten Pool traegt eigene Stimmungs-Tags. Fuer jeden Cocktail wird ein Punktwert gebildet, indem die Gewichte seiner Tags aufsummiert werden. **Der Cocktail mit der hoechsten Punktzahl gewinnt** und wird an die Pumpen geschickt. Wird kein Gesicht erkannt oder tritt ein Fehler auf, wird ein zufaelliger Cocktail aus dem Pool genommen.

Wie beide Teile zusammenspielen

Der springende Punkt, warum die zwei KIs zusammenpassen: Beide sprechen dasselbe **Stimmungs-Vokabular** – eine feste Liste von 21 Tags wie *happy, tropical, dark, classic*. Der Rezept-Generator darf nur Tags aus dieser Liste vergeben, und der Foto-Matcher bewertet genau nach dieser Liste. Dadurch kann der zweite Teil die Cocktails des ersten sinnvoll bewerten.

Die Kette in einem Satz

Zutaten eingeben → LLM erzeugt einen Pool getaggtter Cocktails (mit Sicherheitsnetz Mock) → Spiel laeuft → Selfie des Verlierers wird von ML Kit analysiert → der am besten zur Stimmung passende Cocktail wird ausgewaehlt → seine Pumpenmengen gehen als `mix_a_b_c_d` an die Hardware.

Uebersicht in Stichworten

Fuer die Praesentation – die wichtigsten Punkte auf einen Blick.

Zwei KIs, klar getrennt

- Ein **generativer** Teil: das LLM erfindet Cocktails.
- Ein **analytischer/klassifizierender** Teil: die Bild-KI waehlt einen aus.

Rezept-Generierung (LLM)

- Modell: **Gemma** (~0,5 GB) ueber **flutter_gemma**, laeuft lokal.
- Input: vier Pumpen-Zutaten → Output: 4–6 Cocktails als JSON.
- **Sanitizer** erzwingt sicheres Format: Mengen 0–80 ml, gesamt ≤ 250 ml, min. 2 Pumpen, Tags validiert, Name mit echter Zutat, max. 6 Cocktails.
- **Fallback**: Mock-Generator ohne KI, wenn das Modell fehlt/scheitert.
- Ergebnis wird als Pool gespeichert (ueberlebt Neustart).

Foto-Matching (ML Kit)

- Gesichtsanalyse: Laecheln, Augen, Kopfhaltung + Bild-Labels → Emotion (happy/neutral/sad).
- Emotion → Gewichte pro Stimmungs-Tag.
- Cocktail mit hoechster Tag-Punktzahl gewinnt.
- **Fallback**: Zufallsauswahl, wenn kein Gesicht erkannt wird.

Verbindende Idee & Staerken

- **Gemeinsames Vokabular**: 21 Stimmungs-Tags verbinden Generierung und Auswahl.
- **Alles on-device**: offline, kein API-Key, datenschutzfreundlich (Selfie verlaesst das Handy nie).
- **Robustheit als Design-Prinzip**: beide Teile haben ein Fallback und koennen die App nicht blockieren.
- **Eigentliche Ingenieurleistung**: nicht das LLM „aufrufen“, sondern seine unzuverlaessige Ausgabe durch den Sanitizer hardware-tauglich machen.

Der Sanitizer im Detail

Keine einzelne Funktion, sondern eine **Kette aus kleinen, reinen Funktionen**, die die Modell-Antwort Schritt fuer Schritt saeubern (Datei: `gemma_recipe_parsing.dart`).

Das Grundproblem

- Das LLM soll reines JSON liefern – tut es aber selten sauber.
- Typische Fehler: Fliesstext/Markdown drumherum, erfundene Tags, unrealistische Mengen, fehlende Felder.
- Der Sanitizer zwingt alles zurueck in ein sicheres, hardware-taugliches Format.

Stufe 1 – JSON herausschneiden (`_extractJson`)

- Nimm alles zwischen dem **ersten** „[“ und dem **letzten** „]“ (Fallback: erstes/letztes „{ }“).
- Schneidet Begleittext und ```json-Zaeune automatisch weg.
- Kein JSON gefunden → **FormatException** → Aufrufer schaltet auf den Mock-Generator um.

Stufe 2 – ueber die Liste laufen (`sanitizeCocktails`)

- Akzeptiert ein Array oder ein Objekt mit „cocktails“-Schluessel.
- Nicht-Objekte werden uebersprungen; **max. 6** Cocktails werden behalten.
- Die **id** wird **selbst vergeben** (`gen_<index>_<seed>`) – unabhaengig vom Modell.

Stufe 3 – jedes Feld einzeln saeubern

- **Pumpenmengen (`_sanitizeAmounts`):** (1) jeden Wert auf 0–80 begrenzen, (2) mind. 2 Pumpen erzwingen (leere mit 20 auffuellen), (3) Summe >250 proportional herunterskalieren.
- **Name (`_sanitizeName`):** muss eine echte Zutat enthalten – sonst Hero-Zutat (groesste Menge) voranstellen.
- **Tags (`_sanitizeTags`):** nur die 21 erlaubten `kMoodTags` behalten (Rest verwerfen); leer → Standard „classic“.
- **Beschreibung / Tipp (`_cleanString`):** trimmen; fehlt → automatischer Fallback-Text.

Vorher / Nachher – Beispiel Mengen

Modell liefert `[999, 999, 999, 999]` → Schritt 1 (clamp 0–80) `[80, 80, 80, 80]`
(Summe 320) → Schritt 3 (auf ≤ 250 skalieren) `[62, 62, 62, 62]`. **Das LLM schlaegt vor – der Sanitizer garantiert.**

Warum so gebaut

- **Reine Funktionen, kein Modell noetig** → jeder Schritt ist mit festen Strings testbar (`gemma_sanitizer_test.dart`).
- **Gleicher Vertrag wie der Mock:** hinten kommt immer dasselbe garantierte Format raus (4 Werte, 0–80, Summe ≤ 250 , ≥ 2 Pumpen, gueltige Tags, Name mit Zutat).
- Der Rest der App muss nie zwischen „echt“ und „Fallback“ unterscheiden.